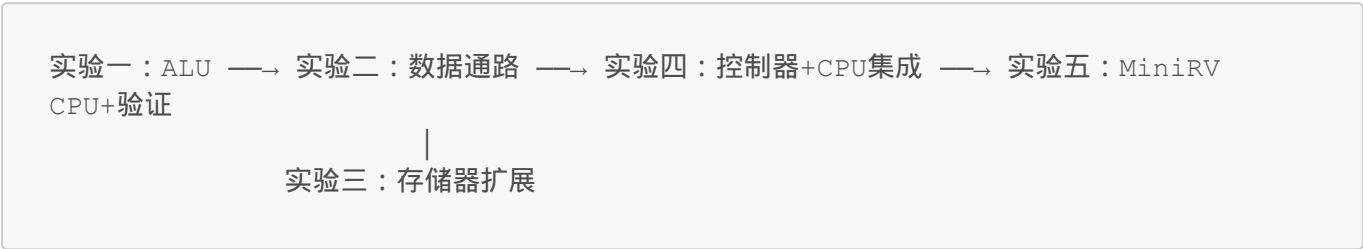


一、实验概述

本项目通过五个递进式实验，从底层 ALU 出发，逐步完成了数据通路部件、存储器扩展、控制器设计，最终实现了一款支持 MiniRV 指令集（37 条指令）的完整单周期 RISC-V 处理器，并通过差分测试框架完成了功能验证。

五个实验的关系如下：



- **实验一（ALU）**：实现 32 位算术逻辑单元，4 种运算 + 4 个标志位
- **实验二（数据通路）**：实现 PC、RegFile、ImmGen、IM、DM 等 7 个核心部件
- **实验三（存储器扩展）**：从 256×8bit 基础单元通过字扩展+位扩展构建 1024×32bit 存储器
- **实验四（控制器）**：实现两级译码控制器，集成 7 指令单周期 CPU
- **实验五（MiniRV）**：升级至 37 条指令，加入字节/半字访存，通过 37 项 Trace 差分测试

二、实验环境

- 主机操作系统：Windows 11
- 服务器操作系统：Ubuntu 24.04
- 开发工具：Xilinx Vivado 2024.2
- 设计语言：SystemVerilog
- 仿真工具：Vivado Simulator (XSim)、Verilator 5.020
- 测试框架：cdp-tests 差分测试框架
- 目标器件：xc7k325tffg900-2

三、实验一：ALU 设计

3.1 设计目标

设计参数化的 32 位 ALU 模块，支持 4 种运算操作，并输出 N/Z/C/V 四个状态标志位。

3.2 ALUControl 编码

ALUControl	功能
00	Add（加法）
01	Sub（减法）
10	And（按位与）

ALUControl	功能
11	Or（按位或）

3.3 标志位设计

标志	含义	计算方式
N	负标志	<code>Result[DATAWIDTH-1]</code>
Z	零标志	<code>Result == 0</code>
C	进位标志	加法：扩展加法进位输出；减法： <code>A >= B</code> （无符号）；逻辑运算：0
V	溢出标志	加法：同号操作数、异号结果则溢出；减法：异号操作数、结果与A异号则溢出

3.4 验证结果

通过穷举测试（4 种运算 $\times 256 \times 256 = 262,144$ 组测试向量），使用 `assert` 自动检查，所有向量通过。

四、实验二：数据通路设计

4.1 设计内容

完成 RISC-V 单周期 CPU 数据通路中全部核心部件的设计，共 8 个模块（含 ALU）。

4.2 各部件概览

模块	文件	类型	关键特性
PC	<code>pc.sv</code>	时序逻辑	异步复位，时钟上升沿更新
Adder	<code>adder.sv</code>	组合逻辑	用于 PC+4 和跳转地址计算
MUX	<code>mux.sv</code>	组合逻辑	参数化位宽的二选一选择器
RegFile	<code>reg_file.sv</code>	混合	组合读 $\times 2$ 、时序写 $\times 1$ ，x0 硬连线为 0
ImmGen	<code>imm_gen.sv</code>	组合逻辑	支持 I/S/B 三种立即数格式
IM	<code>instr_rom.sv</code>	混合	字节地址小端序组装，组合读
DM	<code>data_ram.sv</code>	混合	组合读、时序写，字节级读写，异步复位

4.3 数据通路分析

以 7 条核心指令为例的数据流转分析：

```
算术指令 (add/sub/and/or) :
  IM → RF(rs1, rs2) → ALU → RF(rd写回)
  PC → PC+4 → NPC → PC
```

加载指令 (lw) :

IM → RF(rs1) → ALU + ImmGen → DM(地址) → DM(读出) → RF(rd)

存储指令 (sw) :

IM → RF(rs1→ALU.A, rs2→DM.din) → ALU + ImmGen → DM(地址) → 写入

分支指令 (beq) :

IM → RF(rs1, rs2) → ALU(减) → Zero → PcSrc → MUX → NPC → PC

4.4 验证结果

各模块独立仿真均通过 `$finish`, RF 的 x0 写保护、ImmGen 的三种格式提取、IM/DM 的小端序读写均验证正确。

五、实验三：存储器扩展

5.1 设计目标

从 256×8bit 基础 RAM 单元出发, 通过字扩展和位扩展逐级构建 1024×32bit 存储器。

5.2 层次化设计

```
mini_ram (256×8bit)
  ↓ ×4 片, 字扩展 (2-4 译码器 → CS 片选)
mini_ram_wortext (1024×8bit)
  ↓ ×4 片, 位扩展 (并行字节拆分/拼接)
mini_ram_bitext (1024×32bit)
```

5.3 关键技术

- **字扩展**：高 2 位地址线 `ram_addr_i[9:8]` 通过 2-4 译码器生成 CS[3:0], 选中 1 片工作
- **位扩展**：4 片并行, 写时按字节拆分(`ram_data_i[i*8 +: 8]`), 读时字节拼接
- **generate 语句**：使用 `genvar + generate for` 进行模块阵列例化
- **ena 门控**：`ena && cs[i]` 确保未选中芯片不写入

5.4 验证结果

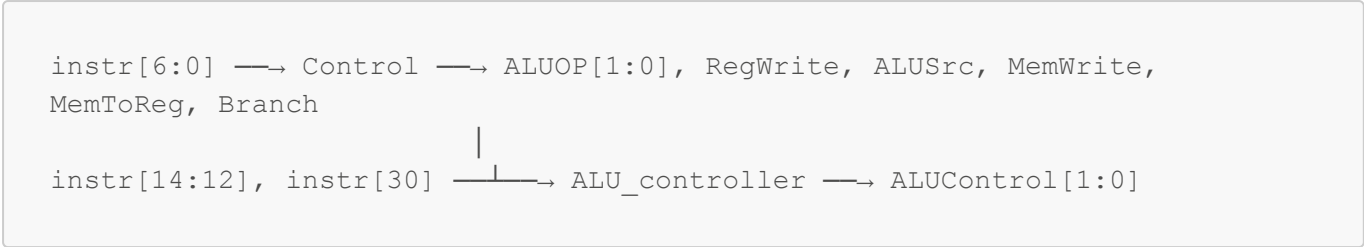
测试平台验证了：基本读写、跨字边界读写、高位地址 (CS 切换)、非覆盖写入、ena 写保护, 全部通过。

六、实验四：控制器与单周期 CPU 集成

6.1 设计内容

实现两级译码控制器, 并将 13 个模块、3 个 MUX 集成完整单周期 CPU, 支持 7 条核心指令。

6.2 两级译码架构



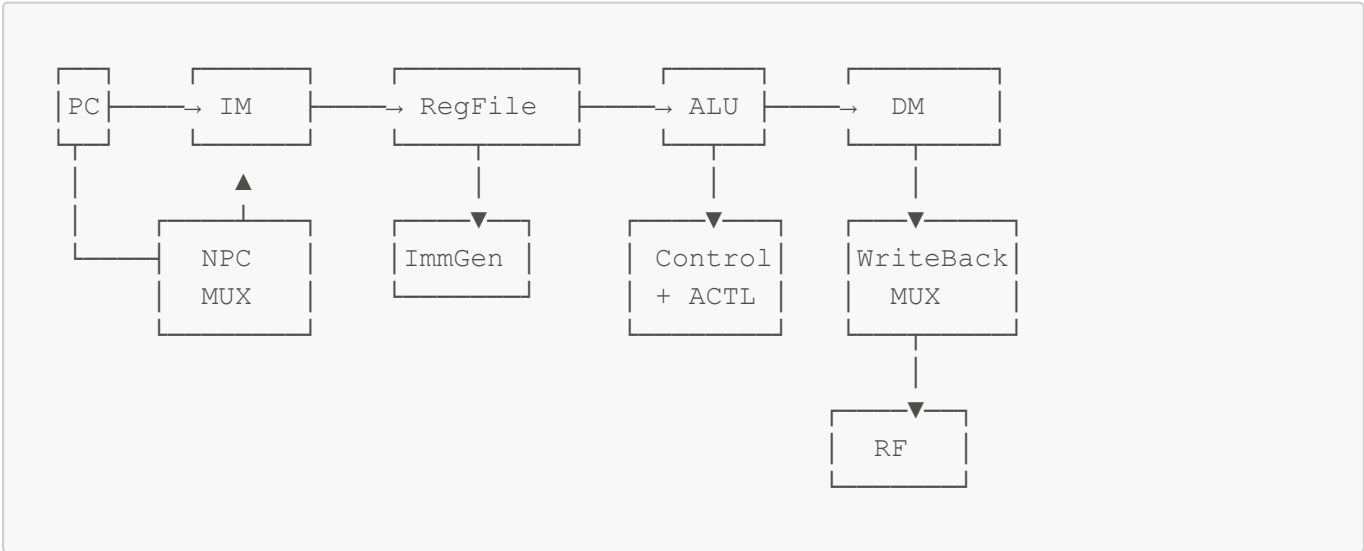
主控制器真值表

指令	opcode	ALUSrc	MemToReg	RegWrite	MemWrite	Branch	ALUOP
R-type	0110011	0	0	1	0	0	10
lw	0000011	1	1	1	0	0	00
sw	0100011	1	X	0	1	0	00
beq	1100011	0	X	0	0	1	01

ALU 控制器译码表

ALUOP	funct7[5]	funct3	指令	ALUControl
00	X	XXX	lw/sw	00 (add)
01	X	XXX	beq	01 (sub)
10	0	000	add	00 (add)
10	1	000	sub	01 (sub)
10	0	111	and	10 (and)
10	0	110	or	11 (or)

6.3 单周期 CPU 架构



模块例化清单（13 个模块 + 3 个 MUX）

模块	实例名	功能
pc	pc_inst	程序计数器
instr_rom	instr_rom_inst	指令存储器
reg_file	reg_file_inst	寄存器堆
imm_gen	imm_gen_inst	立即数生成器
alu	alu_inst	算术逻辑单元
control	control_inst	主控制器
ALU_controller	ALU_controller_inst	ALU 控制器
data_ram	data_ram_inst	数据存储器
pc_add1	adder_left	PC+4 加法器
pc_add2	adder_right	PC+imm 加法器
mux	mux_npc	NPC 地址选择
mux	mux_alusrc	ALU B 端来源选择
mux	mux_dout	写回数据来源选择

关键控制公式：**PcSrc = Branch & Zero**

6.4 验证结果

仿真执行 10 条指令序列（lw ×3, sub, and ×2, sw ×2, or, beq），所有断言通过，验证了 7 条核心指令的正确执行。

七、实验五：MiniRV 指令集 CPU 与验证

7.1 设计目标

将 7 指令核心 CPU 升级为支持 MiniRV 指令集（37 条指令）的完整单周期处理器，通过 Trace 差分测试验证。

7.2 MiniRV 指令集

类别	指令	数量	opcode
R-type	add, sub, and, or, xor, sll, srl, sra, slt, sltu	10	0110011
I-type ALU	addi, andi, ori, xori, slli, srli, srai, slti, sltiu	9	0010011
Load	lb, lbu, lh, lhu, lw	5	0000011
Store	sb, sh, sw	3	0100011
Branch	beq, bne, blt, bltu, bge, bgeu	6	1100011
U-type	lui, auipc	2	0110111 / 0010111

类别	指令	数量	opcode
J-type	jal, jalr	2	1101111 / 1100111

7.3 架构变更

从核心 CPU 到 MiniRV CPU 的主要架构变更：

7.3.1 NPC 重设计

NpcOp	npc 计算	对应指令
00	pc + 4	顺序执行（R/I/S/U 型）
01	isTrue ? pc+offset : pc+4	条件分支（B 型）
10	offset & ~1	jalr
11	pc + offset	jal

新增输出 `pcadd4` 用于 jal/jalr 的链接地址写回。

7.3.2 译码方式变更

从**分级译码**改为**单级译码**：Control 不再输出 ALUOP，ACTL 直接根据 opcode+funct 生成 4-bit ALUControl。

7.3.3 控制信号扩展

信号	原设计	新设计	说明
NpcOp	Branch (1-bit)	2-bit	4 种 NPC 模式
MemToReg	1-bit	2-bit	增加 IMM 和 PC+4 来源
OffsetOrigin	—	1-bit	NPC offset 来源选择
ALUSrcA	—	1-bit	ALU A 端来源选择

7.3.4 ALU 扩展（14 种操作）

ALUControl	操作	代表指令
0000	ADD	add, addi, lw, sw, jalr, auipc
0001	SUB	sub
0010	AND	and, andi
0011	OR	or, ori
0100	XOR	xor, xori
0101	SLL	sll, slli
0110	SRL	srl, srli

ALUControl	操作	代表指令
0111	SRA	sra, srai
1000	EQ	beq
1001	NE	bne
1010	SLT (signed)	blt, slt, slti
1011	SGE (signed)	bge
1100	SLTU (unsigned)	bltu, sltu, sltiu
1101	SGEU (unsigned)	bgeu

7.4 Load/Store 字节半字处理

Load 扩展

DRAM 读取 32-bit 字后，根据 `funct3` 和 `addr_low[1:0]` 提取对应字节/半字：

指令	funct3	操作
lb	000	字节提取 + 符号扩展
lh	001	半字提取 + 符号扩展
lw	010	整字直接通过
lbu	100	字节提取 + 零扩展
lhu	101	半字提取 + 零扩展

Store 合并

由于 DRAM 接口为字级（无字节使能），sb/sh 需读-改-写：从 DRAM 读出旧字，将 rs2 对应字节/半字替换后写回。

7.5 差分测试框架

cdp-tests 框架将 `golden_model`（C 语言 RISC-V 参考模型）与待测 CPU 执行同一指令，逐周期比对 debug 接口的 5 个信号：

信号	说明
debug_wb_have_inst	当前周期是否有指令写回
debug_wb_pc	写回指令的 PC
debug_wb_ena	寄存器写使能
debug_wb_reg	写入的寄存器号
debug_wb_value	写入的寄存器值

调试中修复了两个关键问题：

1. **Debug 时序** — 组合逻辑 debug 信号在时钟沿后反映下条指令。改为 `always_ff` 寄存器化捕获当前指令结果
2. **字节/半字访存** — lb/lbu/lh/lhu 需提取扩展，sb/sh 需读-改-写合并

7.6 测试结果

全部 37 个指令测试通过：

类别	测试数	通过
R-type (add/sub/and/or/xor/sll/srl/sra/slt/sltu)	10	✓
I-type ALU (addi/andi/ori/xori/slli/srli/srai/slti/sltiu)	9	✓
Load (lb/lbu/lh/lhu/lw)	5	✓
Store (sb/sh/sw)	3	✓
Branch (beq/bne/blt/bltu/bge/bgeu)	6	✓
U-type (lui/auipc)	2	✓
J-type (jal/jalr)	2	✓
合计	37	✓

八、总结

本项目通过五个递进式实验，完整经历了一款 RISC-V 单周期 CPU 从底层器件到顶层集成的全流程设计：

实验	核心成果	模块数
ALU	32 位运算单元，4 操作 + 4 标志位	1
数据通路	PC, RF, ImmGen, IM, DM 等核心部件	8
存储器扩展	字扩展+位扩展，256×8→1024×32	3
控制器	两级译码 + 单周期 CPU 集成，7 指令	5（含集成）
MiniRV	架构升级 + 37 指令 + Trace 差分测试	12

核心技术要点总结：

- **参数化设计**：各模块使用 parameter 定义位宽和深度，便于复用
- **时序/组合分离**：RF+DM 采用"组合读+时序写"模式，符合单周期设计需求
- **层次化设计**：从基础单元逐级扩展（存储器实验），模块例化层次清晰
- **系统性方法**：通过数据通路表和控制信号表分析每条指令的部件连接关系
- **差分测试**：与 golden_model 逐周期比对，保证 CPU 行为与 RISC-V 规范一致

从 7 条指令到 37 条指令，从简单 ALU 到完整 CPU，本项目的核心启示在于：**CPU 设计是一个自底向上、逐步抽象的过程**——先理解每条指令的数据流转，再确定所需的部件和连接，最终用控制信号将各部件协调为一个

有机整体。这种从指令集出发、以数据通路表和控制信号表为工具的设计范式，是处理器前端设计的通用方法。